

BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)

Goal:
Reduce search space
by half in every step
 $O(\log n)$ time

1 WHEN TO USE BINARY SEARCH?

- ✓ Data is sorted or can be converted to sorted.
- ✓ We can eliminate half of the search space.
- ✓ Answer looks like → **F F F T T T** (False → True)
- ✓ We can check a condition in $O(1)$ time.
- ✓ Used for exact match, first/last occurrence, or finding answer in search space.

2 CLASSIC BINARY SEARCH TEMPLATE (EXACT MATCH)

```
int left = 0, right = n - 1;
while (left <= right) {
    int mid = left + (right - left) / 2; // safe mid

    if (nums[mid] == target) {
        return mid; // found
    } else if (nums[mid] < target) {
        left = mid + 1; // search right half
    } else {
        right = mid - 1; // search left half
    }
}
return -1; // not found
```

💡 Why $mid = left + (right - left) / 2$?

To avoid integer overflow that can happen in $(left + right) / 2$

3 BINARY SEARCH DECISION TREE

⚡ AHA MOMENT

Binary Search is not about searching an element, it's about finding the boundary where the answer changes

Example:

F	F	F	F	T	T	T	T
---	---	---	---	---	---	---	---

↑
Find the **FIRST T** (transition point)

INTERVIEW TIP

- Always decide what the array is trying to tell you.
- Define the condition, not the element.
- Think: **What am I searching for?**
- Dry run on examples before coding.

❌ COMMON MISTAKES

- Wrong boundaries update (leads to infinite loop)
- Using $mid = (left + right) / 2$ (overflow risk)
- Not handling edge cases (empty array, size 1)
- Wrong condition ($<$ vs $<=$)
- Returning wrong index (first/last occurrence problems)

4 BINARY SEARCH ON SORTED ARRAY (EXAMPLE)

Array: [5, 7, 7, 8, 8, 10] Target = 8

Step 1:

5	7	7	8	8	10
0	1	2	3	4	5

left = 0, right = 5
mid = 2 →
nums[2] = 7
7 < 8, move right

Step 2:

5	7	7	8	8	10
0	1	2	3	4	5

left = 3, right = 5
mid = 4 →
nums[4] = 8
8 == 8, found!

Step 3:

5	7	7	8	8	10
---	---	---	---	---	----

Return index 4

5 COMPLEXITY ANALYSIS

Operation	Complexity
Time Complexity	$O(\log n)$
Space Complexity	$O(1)$ (Iterative)
Best Case	$O(1)$ (if mid is target)
Worst Case	$O(\log n)$

Remember

Every step eliminates half of the search space.

That's the power of Binary Search! 🙌



Master Binary Search Pattern, not just the code.
It helps in 90% of advanced DSA problems.

Page 1 of 4

BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)



GOAL:
Reduce search space by half in every step
 $O(\log n)$ time

Page 2 of 4

6. VARIANTS OF BINARY SEARCH

Variant	When to Use
Classic Binary Search	Exact match in sorted array
First Occurrence	Find first index of target
Last Occurrence	Find last index of target
Search in Rotated Sorted Array	Array is sorted but rotated
Search in Peak Element	Find peak element in bitonic array
Binary Search on Answer	Answer lies in a range, not searching element

7. FIRST & LAST OCCURRENCE TEMPLATES

First Occurrence (Lower Bound)

Find the first index where
 $\text{nums}[\text{mid}] = \text{target}$

```
int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] >= target) {
            if (nums[mid] == target) ans = mid;
            right = mid - 1; // move right
        } else {
            left = mid + 1; // move left
        }
    }
    return ans;
}
```

Last Occurrence (Upper Bound)

Find the last index where
 $\text{nums}[\text{mid}] = \text{target}$

```
int lastOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] <= target) {
            if (nums[mid] == target) ans = mid;
            left = mid + 1; // move left
        } else {
            right = mid - 1; // move right
        }
    }
    return ans;
}
```



PRO TIP: For First occurrence → shrink right when equal
For Last occurrence → shrink left when equal

8. SEARCH IN ROTATED SORTED ARRAY (UNIQUE ELEMENTS)

Example: $\text{nums} = [4, 5, 6, 7, 0, 1, 2]$, $\text{target} = 0$

graph TD
 A["Is nums[mid] == target?"] -- YES --> B["Return mid"]
 A -- NO --> C["Is left half sorted?"]
 C --> D["Left half is sorted"]
 D --> E["Is target in left half?"]
 E -- YES --> F["Is target in right half?"]
 F -- YES --> G["right = mid - 1"]
 F -- NO --> H["left = mid + 1"]
 G --> A
 H --> A

★ Always eliminate the half that is sorted first!

9. BINARY SEARCH ON ANSWER

When to Use?

- Answer is not an element from array.
- We are asked to find minimum / maximum value that satisfies a condition.
- The condition is monotonic (T...T...T or F...F...F).

General Template

```
int left = 0; // minimum possible answer
int right = (int)1e9; // maximum possible answer
int ans = -1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (isValid(mid)) { // condition check
        ans = mid; // store answer
        right = mid - 1; // try for better (smaller) answer
    } else {
        left = mid + 1; // try for bigger answer
    }
}
return ans;
```

Examples

- Minimum days to make M bouquets
- Koko eating bananas
- Allocate minimum pages
- Split array largest sum

10. VISUALIZING BINARY SEARCH (BOUNDARY SEARCH)

Find First Occurrence of target = 4 in $\text{nums} = [1, 2, 4, 4, 4, 5, 6]$

Step 1:

0	1	2	3	4	5	6
1	2	4	4	4	5	6
↑			↑			↑
l = 0			m = 3			r = 6

$\text{nums}[\text{m}] == 4$
 $\text{ans} = 3$
 $\text{r} = \text{m} - 1 = 2$

Step 2:

0	1	2	3	4	5	6
1	2	4	4	4	5	6
↑	↑	↑				
l = 0	m = 1	r = 2				

$\text{nums}[\text{m}] < 4$? no
 $\text{nums}[\text{m}] == 4$
 $\text{ans} = 1$
 $\text{r} = \text{m} - 1 = 0$

Step 3:

0	1	2	3	4	5	6
1	2	4	4	4	5	6
↑						
l=0 m=0						
r=0						

$\text{nums}[\text{m}] < 4$
 $\text{l} = \text{m} + 1 = 1$
Loop stops ($\text{l} > \text{r}$)

Answer = 1 (first index of 4)

**REMEMBER**

Binary Search is all about reducing the search space.
The key is to think about the answer, not the element!

11. COMMON MISTAKES & HOW TO AVOID THEM

Mistake	Why It's Wrong	Fix
✗ Using $(\text{left} + \text{right}) / 2$	Can overflow when left and right are large	Use <code>left + (right - left) / 2</code>
✗ Incorrect boundary update	Can cause infinite loop or wrong result	Be careful with <code>left = mid + 1</code> or <code>right = mid - 1</code>
✗ Not handling edge cases	Empty array, size 1 array, target not found	Always test edge cases
✗ Confusing first & last occurrence	Logic difference in shrinking	Remember: first → shrink right last → shrink left
✗ Wrong condition in rotated array	Leads to eliminating wrong half	Check which half is sorted first

**INTERVIEW GOLDEN RULE**

Dry run on paper. Visualize.
Write clean, bug-free code. Handle all edge cases.



BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)



GOAL:
Reduce search space
by half in every step
 $O(\log n)$ time

Page 3 of 4

12. BOUNDARY SEARCH VISUALIZATION

We are looking for **FIRST** occurrence of 8

Array:	1	2	4	5	7	8	8	8	8	10	13
	0	1	2	3	4	5	6	7	8	9	10

Step	left	right	mid	nums[mid]	Action
1	0	10	5	8	ans = 5, move left (right = 4)
2	0	4	2	4	nums[mid] < 8, move right (left = 3)
3	3	4	3	5	nums[mid] < 8, move right (left = 4)
4	4	4	4	7	nums[mid] < 8, move right (left = 5)
5	5	4	-	-	left > right, stop

✓ **Answer = 5 (First index of 8)**

**Key Idea:**

When `nums[mid] == target`, don't return immediately. Store the answer and keep searching in the left half.

13. MORE BINARY SEARCH TEMPLATES

Lower Bound (First $\geq$ target)

```
int left = 0, right = n;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] >= target)
        right = mid;
    else
        left = mid + 1;
}
return left;
```

Returns:

First index where
`nums[index] >= target`

If none exists, returns n

Use Case:

- Find insertion position (upper bound logic)
- Longest Increasing Subsequence (LIS)

Upper Bound (First $>$ target)

```
int left = 0, right = n;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] > target)
        right = mid;
    else
        left = mid + 1;
}
return left;
```

Returns:

First index where
`nums[index] > target`

If none exists, returns n

Use Case:

- Count elements $\leq$ target
- Find last occurrence + 1

Floor (Last $\leq$ target)

```
int left = 0, right = n - 1,
ans = -1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] <= target) {
        ans = mid;
        left = mid + 1;
    } else right = mid - 1;
}
return ans;
```

Returns:

Last index where
`nums[index] <= target`

If none exists, returns -1

Use Case:

- Find floor value (greatest $\leq$ target)

14. BINARY SEARCH ON 2D MATRIX (ROW-WISE SORTED)

Example Matrix:

1	3	5	7
10	11	16	20
23	30	34	50
60	65	70	80

Search = 34

Approach:

1. Treat matrix as 1D array.
2. Index mapping:

```
row = mid / m;
col = mid % m;
```

3. Apply normal Binary Search.

Code:

```
int row = matrix.length, col = matrix[0].length;
int left = 0, right = row * col - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    int r = mid / col, c = mid % col;
    if (matrix[r][c] == target) return true;
    else if (matrix[r][c] < target) left = mid + 1;
    else right = mid - 1;
}
return false;
```



Works because each row is sorted and first element of a row is greater than last of previous row.

16. SEARCH IN INFINITE / VERY LARGE ARRAY

Problem:

Array is sorted but size is unknown or very large.

Solution:

1. Find the range (low, high) where target can exist.
2. Apply Binary Search in that range.

1. Expand range:

```
low = high + 1;
high = high * 2;
```

2. Apply normal Binary Search between low and high.

Target = 15

...	1	3	7	9	12	15	18	2
	0	1	2	3	4	5	6	7

↑ low = 4 Expanding Range ↑ high = 7

until `arr[high] >= target`
or `arr[high]` is out of bounds

✓ Time Complexity: $O(\log \text{index_of_target})$

15. SEARCH IN PEAK ELEMENT (BITONIC ARRAY)

Bitonic Array (First increasing then decreasing)



Steps:

1. Find the peak element index.
2. If `target == peak` → return true
3. If `target < peak` → Binary Search in left (increasing part)
4. If `target > peak` → Binary Search in right (decreasing part)

Find Peak Index

```
int l = 0, r = n - 1;
while (l < r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] < arr[mid + 1])
        l = mid + 1;
    else
        r = mid;
}
return l;
```

Binary Search in Decreasing Order

```
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == target)
        return true;
    else if (arr[mid] > target)
        l = mid + 1;
    else
        r = mid - 1;
}
```

17. REAL INTERVIEW QUESTIONS (POPULAR)

1. Binary Search
2. Search Insert Position
3. Find First and Last Position of Element in Sorted Array
4. Search in Rotated Sorted Array
5. Search in Rotated Sorted Array II (duplicates)
6. Koko Eating Bananas
7. Minimum Eating Speed
8. Ship Packages Within D Days
9. Allocate Minimum Number of Pages
10. Split Array Largest Sum
11. Find Peak Element
12. Find Minimum in Rotated Sorted Array



Most questions are variations of

- Boundary Search
- Search on Answer
- Rotated Array Search



KEY TAKEAWAY

Binary Search is about thinking smart, defining the search space correctly, and shrinking it efficiently.



REMEMBER

- Think of the question type first.
- Choose the correct template.
- Handle edge cases carefully.

TIME COMPLEXITY

$O(\log n)$

SPACE COMPLEXITY

$O(1)$
(Iterative)

BINARY SEARCH

FAANG INTERVIEW CHEAT SHEET (JAVA)



GOAL:

Reduce search space by half in every step
 $O(\log n)$ time

Page 4 of 4

18. TIME & SPACE COMPLEXITY SUMMARY

Problem Type	Time Complexity	Space Complexity	Notes
Classic Binary Search	$O(\log n)$	$O(1)$	Iterative
First / Last Occurrence	$O(\log n)$	$O(1)$	Extra checks
Search on Answer	$O(\log N \times \log M)$	$O(1)$	N = array size, M = answer range
Search in 2D Matrix	$O(\log (m \times n))$	$O(1)$	Treat as 1D array
Peak Element	$O(\log n)$	$O(1)$	Bitonic array
Rotated Sorted Array	$O(\log n)$	$O(1)$	Unique elements

20. BINARY SEARCH FLOW SUMMARY

graph TD
 A[Is the array sorted?] -->|YES| B[What are we looking for?]
 A -->|NO| C[Can we sort it?]
 B --> D[Exact Element]
 B --> E[First / Last Occurrence]
 B --> F[Min / Max Satisfying Condition]
 C --> D1[Classic Binary Search]
 E --> E1[First / Last Templates]
 F --> F1[Binary Search on Answer]
 C1[Sort + Classic Binary Search] -->|YES| G[Sort + Classic Binary Search]
 C1 -->|NO| H[Binary Search Not Applicable]

style A fill:#fff,stroke:#1d4ed8,stroke-width:2px style B fill:#fff,stroke:#1d4ed8,stroke-width:2px style C fill:#fff,stroke:#1d4ed8,stroke-width:2px style D fill:#fff,stroke:#1d4ed8,stroke-width:2px style E fill:#fff,stroke:#1d4ed8,stroke-width:2px style F fill:#fff,stroke:#1d4ed8,stroke-width:2px style D1 fill:#fff,stroke:#1d4ed8,stroke-width:2px style E1 fill:#fff,stroke:#1d4ed8,stroke-width:2px style F1 fill:#fff,stroke:#1d4ed8,stroke-width:2px style C1 fill:#fff,stroke:#1d4ed8,stroke-width:2px style G fill:#fff,stroke:#1d4ed8,stroke-width:2px style H fill:#fff,stroke:#1d4ed8,stroke-width:2px

19. PATTERN RECOGNITION GUIDE

Look at the problem and identify the pattern:

Exact value in sorted array	→	Classic Binary Search
First / Last position of a target	→	First / Last Occurrence Templates
Smallest / Largest value satisfying a condition	→	Binary Search on Answer (Optimization Problems)
Matrix is sorted (row-wise & col-wise)	→	Treat as 1D Binary Search
Array is rotated sorted	→	Find pivot, then Binary Search in correct half
Bitonic / Mountain Array	→	Find peak, then search in increasing / decreasing part

21. BINARY SEARCH SUMMARY (KEY POINTS)

- ✓ Always think: Can I eliminate half of the search space?
- ✓ Define search space correctly.
- ✓ Choose the right template (classic / first / last / answer).
- ✓ Handle boundaries and edge cases.
- ✓ Avoid infinite loops by correct pointer updates.
- ✓ Dry run with small examples before coding.
- ✓ Write clean, bug-free, and well-commented code.



REMEMBER:

Binary Search is all about reducing uncertainty in the smartest way possible!

22. COMMON EDGE CASES• **Array with 1 element**

```
nums = [5], target = 5 → return 0
```

• **Target not present**

```
nums = [1,2,3,4], target = 10 → return -1
```

• **All elements same**

```
nums = [7,7,7,7], target = 7 → return first / last as per need
```

• **First element target**

```
nums = [1,2,3,4], target = 1 → return 0
```

• **Last element target**

```
nums = [1,2,3,4], target = 4 → return n-1
```

• **Check for overflow in mid calculation**

```
int mid = left + (right - left) / 2; // safe
```

23. DRY RUN TIPS

- Always write left, right, mid.
- Highlight the mid element.
- Write the comparison result.
- Update the pointers.
- Repeat until condition breaks.
- Finally, write the answer.



Pro Tip:
Practice dry run without coding first!
It improves your problem-solving speed.

24. QUICK REFERENCE (JAVA)

Mid (Safe)	<code>int mid = left + (right - left) / 2;</code>
Left Half	<code>right = mid - 1;</code>
Right Half	<code>left = mid + 1;</code>
Loop Condition	<code>while (left <= right)</code>
Return Not Found	<code>return -1;</code>
First Occurrence	<code>right = mid - 1;</code> // when equal
Last Occurrence	<code>left = mid + 1;</code> // when equal
Lower Bound	<code>right = mid - 1;</code> // when equal
Upper Bound	<code>left = mid + 1;</code> // when equal

25. FAANG INTERVIEW CHECKLIST

- | | |
|-------------------------------------|------------------------------------|
| ✓ Understand problem & constraints | ✓ Check time & space complexity |
| ✓ Identify if Binary Search applies | ✓ Test with custom cases |
| ✓ Choose correct template | ✓ Explain your approach clearly |
| ✓ Define search space | ✓ Be ready for follow-up questions |
| ✓ Handle edge cases | ✓ Optimize if required |
| ✓ Dry run with examples | |
| ✓ Write clean code | |

**GOLDEN RULE**

If you can solve it by eliminating half every time, think Binary Search first!

26. MOTIVATION BOOSTER

- ★ Binary Search is a superpower.
- ★ It turns $O(n)$ into $O(\log n)$.
- ★ Master this pattern, crack any interview!



YOU'VE GOT THIS! 🙌



MASTER BINARY SEARCH PATTERN, NOT JUST THE CODE.



THINK → ELIMINATE → NARROW → FIND
That's the Binary Search Way!

Practice. Analyze. Optimize.
Become Unstoppable!